TP02: How to Work with Big Data Files (5GB+)

Rapport Technique

University of Eloued - Big Data Analytics

GHRIBI ACHOIK
Groupe 1 - Master 2 IA & Data Science
Season: 2026/2027

October 14, 2026

Abstract

Abstract

Ce rapport présente une analyse comparative de trois méthodes pour le traitement d'un fichier CSV de 5.33 GB (574,945 lignes, 1035 colonnes) : **Pandas traditionnel**, **Pandas Chunking**, et **Dask DataFrame**. L'expérimentation a été menée dans **Visual Studio Code** sur une machine locale (Windows 10, 8 GB RAM, 4 cores, HDD).

Les résultats montrent que :

- **Pandas échoue** avec `ParserError: out of memory` après 160.05s.
- **Chunking réussit** en 104.93s avec une RAM stable (30-43%).
- **Dask réussit** en 270.96s avec la RAM la plus basse (36.2%).

Conclusion principale : Sur notre machine (HDD, 8 GB RAM, 4 cores), **Chunking est la méthode la plus efficace** car elle combine rapidité et stabilité mémoire.

Mots-clés : Big Data, Pandas, Dask, Chunking, OOM, Lazy Evaluation, Partitions, HDD.

Contents

1 Introduction **4**
1.1 Objectifs . . . 4
1.2 Environnement . . . 4

2 Dataset **4**

3 Task 1 : Échec Traditionnel (OOM) **4**
3.1 Objectif . . . 4
3.2 Code . . . 4
3.3 Résultats . . . 5
3.4 Analyse et Explication . . . 5

4 Task 2 : Pandas Chunking **5**
4.1 Objectif . . . 5
4.2 Code . . . 6
4.3 Résultats . . . 6
4.4 Analyse et Explication . . . 6

5 Task 3 : Dask DataFrame **7**
5.1 Objectif . . . 7
5.2 Code . . . 7
5.3 Résultats . . . 7
5.4 Analyse et Explication . . . 8

6 Comparaison Finale **8**
6.1 Conformité aux attentes . . . 8

7 Pourquoi Chunking est la Meilleure Méthode ? **8**
7.1 1. Le Plus Rapide . . . 9
7.2 2. RAM Stable . . . 9
7.3 3. Adapté au HDD . . . 9
7.4 4. Simple . . . 9
7.5 5. Pas d'Overhead . . . 9
7.6 6. Fiable . . . 9

8 Conclusions **9**
8.1 Résumé . . . 9
8.2 Recommandations . . . 10
8.3 Conclusion Générale . . . 10

1 Introduction

1.1 Objectifs

1. Comprendre les limites du chargement traditionnel (OOM).
2. Implémenter la technique de **Chunking** avec Pandas.
3. Utiliser **Dask DataFrame** pour le calcul parallèle et différé.
4. **Déterminer la méthode optimale** pour notre environnement.

1.2 Environnement

Composant	Valeur
Éditeur	Visual Studio Code
OS	Windows 10
RAM	8 GB (7.9 GB usable)
CPU	4 cores
Disque	HDD
Python	3.14

Table 1: Environnement d'exécution

2 Dataset

Caractéristique	Valeur
Source	Kaggle - CMI Precompute Features
Fichier	train.csv
Taille	5.33 GB
Lignes	574,945
Colonnes	1035
Licence	MIT

Table 2: Spécifications du dataset

3 Task 1 : Échec Traditionnel (OOM)

3.1 Objectif

Tenter de lire le fichier avec `pd.read_csv()` standard et documenter l'erreur OOM.

3.2 Code

Listing 1: task1_oom.py

```
import pandas as pd
import psutil
import time

print("TASK 1: Traditional Failure (OOM Challenge)")

```

```
mem = psutil.virtual_memory()
print(f"RAM Before: {mem.percent}%")

start = time.time()
try:
    df = pd.read_csv("train.csv", low_memory=False)
    print(f"Loaded in {time.time()-start:.2f}s")
except Exception as e:
    print(f"Error after {time.time()-start:.2f}s")
    print(f"{type(e).__name__}: {e}")

mem = psutil.virtual_memory()
print(f"RAM After: {mem.percent}%")
```

3.3 Résultats

```
RAM Before: 55.3% (4.4 GB / 7.9 GB)

Attempting to load train.csv with pd.read_csv()...

Error after 160.05s: ParserError
   Error tokenizing data. C error: out of memory

RAM After: 32.1% (2.5 GB)

CONCLUSION: Standard pandas cannot handle 5GB files!
```

3.4 Analyse et Explication

Métrique	Valeur
RAM avant	55.3% (4.4 GB)
RAM après	32.1% (2.5 GB)
Temps	160.05s
Erreur	ParserError: out of memory
Statut	Échec (OOM)

Pourquoi cette erreur ?

- Avec `low_memory=False`, Pandas charge tout le fichier en RAM d'un seul coup.
- Avec 1035 colonnes et 574,945 lignes, le fichier nécessite plus de 15 GB en RAM.
- La machine ne dispose que de 7.9 GB.
- Le parser C échoue alors avec `out of memory`.

Conclusion : Pandas traditionnel **n'est pas adapté** aux fichiers $>$ 5 GB sur une machine avec 8 GB RAM.

4 Task 2 : Pandas Chunking

4.1 Objectif

Lire le fichier par chunks de 100,000 lignes pour maintenir une RAM constante.

4.2 Code

Listing 2: task2_chunking.py

```python
import pandas as pd
import psutil
import time

CHUNK_SIZE = 100_000

start = time.time()
total_rows = 0
n_chunks = 0

for chunk in pd.read_csv("train.csv", chunksize=CHUNK_SIZE):
    n_chunks += 1
    total_rows += len(chunk)
    mem = psutil.virtual_memory()
    print(f"Chunk {n_chunks} | Rows: {total_rows:,} | "
          f"RAM: {mem.percent}% | Time: {time.time()-start:.1f}s")

print(f"Total: {total_rows:,} rows in {time.time()-start:.2f}s")
```

4.3 Résultats

```
RAM Before: 30.6%

Chunk 1 | Rows: 100,000 | RAM: 41.8% | Time:  17.2s
Chunk 2 | Rows: 200,000 | RAM: 41.4% | Time:  34.6s
Chunk 3 | Rows: 300,000 | RAM: 41.3% | Time:  52.3s
Chunk 4 | Rows: 400,000 | RAM: 41.4% | Time:  70.0s
Chunk 5 | Rows: 500,000 | RAM: 42.7% | Time:  90.5s
Chunk 6 | Rows: 574,945 | RAM: 40.3% | Time: 104.9s

Chunking Complete!
   Total rows: 574,945
   Time: 104.93s
   RAM: 40.3%
```

4.4 Analyse et Explication

Métrique	Valeur
Chunks	6
Total rows	574,945
Temps	104.93s
RAM min	40.3%
RAM max	42.7%
Variation	2.4% (constante)
Statut	Succès

Pourquoi ce succès ?

- Pandas lit uniquement 100,000 lignes à la fois.
- Après traitement, la mémoire de ce chunk est **libérée**.

- Le chunk suivant est ensuite lu.
- **Résultat :** RAM constante à ∼40%.

Conclusion : Chunking **est très efficace** : il combine rapidité et stabilité mémoire. C'est la méthode la plus adaptée à notre machine.

5 Task 3 : Dask DataFrame

5.1 Objectif

Utiliser Dask pour le calcul parallèle avec Lazy Evaluation.

5.2 Code

Listing 3: task3_dask.py

```python
import dask.dataframe as dd
import psutil
import time

ddf = dd.read_csv("train.csv", blocksize="2GB")
print(f"Partitions: {ddf.npartitions}")

total_rows = ddf.shape[0]

compute_start = time.time()
rows_result = total_rows.compute()
compute_time = time.time() - compute_start

print(f"Total rows: {rows_result:,}")
print(f"Compute time: {compute_time:.2f}s")
```

5.3 Résultats

```
RAM Before: 33.1% (2.61 GB)

[LAZY] Creating Dask DataFrame...
   Done in 0.305s
   Partitions: 2
   RAM: 33.2%

[LAZY] Building graph...

[COMPUTE] Now executing...

Dask Complete!
   Total rows:  574,945
   Compute time: 270.96s
   Total time:   271.38s
   RAM: 36.2%
```

5.4 Analyse et Explication

Métrique	Valeur
Partitions	2
Lazy setup	0.305s
Compute time	270.96s
RAM min	33.1%
RAM max	36.2%
Statut	Succès

Pourquoi ce résultat ?

- **RAM :** la plus basse grâce à Lazy Evaluation (planification avant exécution).
- **Temps :** plus long que Chunking car :
  - Le disque HDD est lent.
  - Seulement 4 cœurs CPU.
  - Overhead de gestion des 2 partitions.

Conclusion : Dask est excellent pour la RAM, mais pas optimal sur HDD. Idéal pour SSD + 16+ cœurs.

6 Comparaison Finale

Méthode	RAM	Temps	Statut
Standard Pandas	100%	160s	Échec (OOM)
Pandas Chunking	40-43%	**104.93s**	Succès
Dask DataFrame	**36.2%**	270.96s	Succès

Table 3: Comparaison des performances

6.1 Conformité aux attentes

Attendu	Obtenu	Conforme
Pandas: 100% RAM, Crash	OOM	Oui
Chunking: Low RAM	40-43%	Oui
Dask: Low RAM	36.2%	Oui
Dask: Fast	271s (lent)	Partiel

7 Pourquoi Chunking est la Meilleure Méthode ?

Sur la base des résultats, **Chunking** est la méthode la plus adaptée à notre environnement (HDD, 8 GB RAM, 4 cores).

7.1 1. Le Plus Rapide

Méthode	Temps	Différence
Chunking	**104.93s**	—
Dask	270.96s	+166s (+158%)

Chunking est **2.5 fois plus rapide** que Dask.

7.2 2. RAM Stable

Méthode	RAM min	RAM max
Chunking	40.3%	42.7%
Dask	33.1%	36.2%

Chunking maintient une RAM **stable** (variation 2.4%). Idéal pour éviter les surprises.

7.3 3. Adapté au HDD

Sur un HDD :

- **Chunking** : lecture séquentielle optimale.
- **Dask** : multi-partitions = beaucoup de seek time.

7.4 4. Simple

- Chunking : 3 lignes de code.
- Dask : configuration + Lazy + compute.

7.5 5. Pas d'Overhead

Dask doit gérer les partitions et combiner les résultats. Chunking n'a aucun overhead.

7.6 6. Fiable

Chunking donne des résultats **reproductibles** et **prévisibles**.

8 Conclusions

8.1 Résumé

- **Pandas :** Échoue (OOM) à cause du chargement complet.
- **Chunking :** Réussit rapidement avec RAM stable.
- **Dask :** RAM minimale mais lent sur HDD.

8.2 Recommandations

1. **Fichiers $<$ 1 GB :** Pandas standard.
2. **Fichiers 1-10 GB : Chunking** (meilleur compromis).
3. **Fichiers $>$ 10 GB :** Dask avec SSD.
4. **Clusters :** Dask.

8.3 Conclusion Générale

Pour notre environnement (HDD, 8 GB RAM, 4 cores), **Chunking est la méthode optimale**. Elle offre le meilleur compromis entre **vitesse**, **stabilité mémoire**, et **simplicité**. Dask est plus économe en RAM mais souffre du HDD.

Références

- Kaggle : https://www.kaggle.com/datasets/urvishp80/cmi-precompute-features
- Pandas Docs : https://pandas.pydata.org/docs/
- Dask Docs : https://docs.dask.org/
- VS Code : https://code.visualstudio.com/